

МЕТОДИЧЕСКИЕ УКАЗАНИЯ

для студентов по выполнению проектной работы по дисциплине

«Технологии хранения и обработки больших данных»

Методические указания предназначены для проведения проектной работы в рамках расчетно-графического задания по дисциплине «Технологии хранения и обработки больших данных». Целью настоящих методических указаний является обучение студентов практическим навыкам от проектирования архитектуры хранения и сбора данных из различных источников до построения прогностической модели и оценки качества всего решения, с использованием современных технологий больших данных и машинного обучения.

Методические указания составлены в соответствии с требованиями программы дисциплины «Технологии хранения и обработки больших данных» для студентов специальности 09.03.02 «Информационные системы и технологии» и утверждены на заседании кафедры _____ протокол № _____ от _____.20__.

ЭТАПЫ ВЫПОЛНЕНИЯ ПРОЕКТА

Этап 1. Формулировка бизнес-задачи с использованием генеративного ИИ

Задачи:

1. Используя генеративный ИИ (ChatGPT, YandexGPT, GigaChat), сформулировать бизнес-задачу для выбранной предметной области.
2. С помощью ИИ провести анализ предметной области: определить ключевые сущности, процессы и проблемы.
3. Определить требования к данным (объем, источники, структура, периодичность обновления) для решения задачи.
4. Спроектировать архитектуру инфраструктуры Big Data, включая:
 - Источники данных (из внешних систем, файлы, базы данных, данные в реальном времени).
 - Хранилище сырых данных (Data Lake / HDFS / MinIO).
 - Система хранения структурированных данных (PostgreSQL, ClickHouse).
 - Система хранения неструктурированных данных (MongoDB, JSONB).
 - Обработка данных (ETL/ELT на Spark).
 - Оркестрация конвейеров (Apache Airflow).
 - Data Lineage и качество данных (OpenMetadata).
 - ML-модель (задача классификации/регрессии/рекомендаций).
5. Обосновать выбор технологий.

Результат:

- Архитектурная схема (диаграмма потоков данных).
- Обоснование выбора технологического стека (таблица сравнения альтернатив).
- План-график работ.

Компетенции: BD-4.1 (выбор технологий), BD-5.1 (выбор стека), SS-2.1 (командное планирование).

Этап 2. Проектирование модели хранения данных

Задачи:

1. Разработать схему реляционной базы данных (PostgreSQL):
 - Спроектировать таблицы, связи, индексы.
 - Обосновать выбор нормализации/денормализации.
2. Разработать модель для неструктурированных данных (MongoDB или JSONB):
 - Определить структуру документов.

- Обосновать выбор.
- 3. Спроектировать структуру озера данных (HDFS / MinIO / S3):
 - Определить иерархию каталогов (raw, transformed, curated, feature_store).
 - Выбрать формат хранения (Parquet, ORC, Avro) и обосновать.
- 4. Разработать схему Feature Store (хранилище признаков для ML-модели).

Результат:

- ER-диаграмма для реляционной базы данных.
- Примеры документов для MongoDB/JSONB.
- Схема каталогов Data Lake.
- Описание схемы Feature Store.

Компетенции: BD-3.1 (разработка решений с применением различных технологий хранения), BD-5.1 (выбор стека).

Этап 3. Сбор и подготовка данных

Задачи:

1. **Сбор данных:**
 - Реализовать скрипты для сбора данных из выбранных источников.
 - Загрузить сырые данные в озеро данных (MinIO/HDFS).
2. **Первичная обработка:**
 - Реализовать предварительную очистку данных на Spark (удаление дубликатов, обработка пропусков, нормализация форматов).
 - Сохранить очищенные данные (слой transformed).

Результат:

- Скрипты сбора данных (Python).
- Spark-скрипты для очистки данных.

Компетенции: BD-2.2 (сбор и подготовка данных), PL-1.3 (разработка на Python).

Этап 4. Разработка ETL/ELT-процесса

Задачи:

1. **Реализация трансформаций на Spark:**
 - Агрегация данных.
 - Инженерия признаков для ML-модели (не менее 5 признаков).
 - Преобразование данных в формат, готовый для загрузки в хранилища.
2. **Загрузка данных в целевые хранилища:**
 - В реляционную БД (PostgreSQL) – структурированные данные.
 - В NoSQL-хранилище (MongoDB/JSONB) – неструктурированные данные.
 - В Feature Store (отдельная таблица/папка с признаками).
3. **Оркестрация конвейеров в Apache Airflow:**
 - Создать DAG с последовательностью задач:
Сбор данных → Очистка → Трансформация → Загрузка в PostgreSQL → Загрузка в MongoDB → Обновление Feature Store.
 - Настроить расписание выполнения (например, ежедневно).
 - Добавить обработку ошибок и логирование.
4. **Настройка Data Lineage (OpenMetadata):**
 - Настроить отслеживание происхождения данных.
5. **Использование генеративного ИИ для проверки кода:**
 - Провести проверку кода ETL-процессов с использованием генеративного ИИ.
 - Исправить выявленные ошибки.
 - Отобразить результаты проверки в отчете.

Результат:

- Код Spark-приложений для трансформации.

- DAG для Airflow.
- Скриншоты интерфейса OpenMetadata с Data Lineage.

Компетенции: BD-4.1 (выбор и применение технологий обработки), BD-4.3 (тестирование качества решений), PL-1.3 (разработка на Python).

Этап 5. Контроль качества данных и оптимизация

Задачи:

- Внедрение контроля качества данных (Data Quality, DQ):**
 - На этапе Extract: проверка наличия обязательных полей, типов данных.
 - На этапе Transform: проверка на NULL-значения, выбросы, уникальность.
 - На этапе Load: проверка количества записей, целостности ссылок.
 - Создание DQ-отчета (количество отклоненных записей, причины).
- Оптимизация производительности:**
 - Проанализировать узкие места в процессе (Spark UI, логи Airflow).
 - Применить оптимизации: индексы в PostgreSQL, партиционирование, кэширование в Spark.
 - Зафиксировать улучшение времени выполнения.
- Оценка качества ML-модели:**
 - Обучить ML-модель на данных из Feature Store.
 - Оценить качество модели (метрики: ROC-AUC, F1, Precision, Recall).
 - Проанализировать, как качество данных влияет на качество модели.

Результат:

- Код DQ-проверок.
- DQ-отчет.
- Отчет об оптимизации (время до/после).
- Метрики ML-модели.

Компетенции: BD-4.3 (тестирование и оценка качества решений), PL-1.3 (разработка на Python).

Этап 6. Документирование и защита проекта

Задачи:

1. Подготовить итоговый отчет по проекту.
2. Подготовить презентацию (10-15 минут).
3. Провести демонстрацию работающего прототипа.

Результат:

- Отчет.
- Презентация.
- Демонстрация прототипа.

Компетенции: SS-2.1 (эффективная коммуникация, представление результатов).

Необходимы ресурсы

Компонент	Рекомендуемые технологии
Хранилище озера данных	MinIO (локально), HDFS (в кластере), S3 (облако)
Реляционная СУБД	PostgreSQL
NoSQL-хранилище	MongoDB, Cassandra (для колоночных данных), Redis (для кэша)

Компонент	Рекомендуемые технологии
Форматы файлов	Parquet (для аналитики), ORC, Avro
Обработка данных	Apache Spark (PySpark), Trino (для интерактивных запросов)
Оркестрация	Apache Airflow
Data Lineage / DQ	OpenMetadata, Apache Atlas, Great Expectations
Разметка данных	CVAT, Label Studio, Labelbox
Язык программирования	Python (PySpark, Pandas, SQLAlchemy)
Контейнеризация	Docker, Docker Compose
Версионирование кода	Git, GitHub/GitLab
ML-библиотеки	scikit-learn, XGBoost, LightGBM, CatBoost
Визуализация	Jupyter Notebook, Matplotlib, Seaborn, Superset

Данная проектная работа объединяет все изученные в рамках дисциплины технологии и методы в единое комплексное решение. Проект является **итоговой аттестацией** по дисциплине и позволяет оценить уровень сформированности компетенций Инженера по данным.

Примерная структура репозитория

```
project-big-data/
├── README.md                # Описание проекта
├── docker-compose.yml       # Поднятие инфраструктуры (PostgreSQL, MinIO, Spark, Airflow)
├── airflow/
│   └── dags/
│       └── project_dag.py   # DAG для оркестрации пайплайна
├── spark/
│   ├── ingestion.py        # Сбор данных
│   ├── cleaning.py         # Очистка данных
│   ├── transformation.py   # Трансформация и инжинерия признаков
│   └── load.py              # Загрузка в хранилища
├── scripts/
│   ├── generate_data.py    # Генерация синтетических данных
│   ├── dq_checks.py        # Проверки качества данных
│   └── openmetadata_config.py # Настройка Data Lineage
├── model/
│   ├── train.py            # Обучение ML-модели
│   └── evaluate.py         # Оценка модели
├── docs/
│   ├── architecture.md    # Архитектурная схема и обоснование
│   ├── er_diagram.png     # ER-диаграмма
│   ├── data_lake_schema.md # Структура озера данных
│   ├── annotation_guideline.md # Инструкция для разметчиков
│   ├── dq_report.md       # DQ-отчет
│   └── project_report.pdf  # Итоговый отчет
└── data/
    ├── raw/                # Сырые данные (не хранить в репозитории)
    ├── transformed/        # Очищенные данные
    ├── curated/            # Агрегированные данные
    └── feature_store/      # Признаки для ML
```

Требования к README.md:

1. Название проекта и состав команды.
2. Описание предметной области и задачи.
3. Инструкция по развертыванию (docker-compose up).
4. Структура репозитория.
5. Ссылки на документацию.

Часто задаваемые вопросы

Вопрос: Что делать, если данные слишком большие для локального компьютера?

Ответ: Используйте усеченный датасет (sample) для разработки и отладки. Полный датасет можно обрабатывать на облачной платформе (Yandex Cloud, AWS) или демонстрировать на защите с помощью скриншотов.

Вопрос: У нас нет данных для разметки. Можно ли пропустить этот этап?

Ответ: Если в вашей задаче разметка не требуется (например, прогнозирование временных рядов или биржевые данные), этот этап можно пропустить с обоснованием в отчете.

Вопрос: Какие объемы данных считаются "достаточными" для проекта?

Ответ: Рекомендуемый минимальный объем: 100 000 записей для обработки и 10 000 записей для ML-модели (сбалансированных по классам).

Вопрос: Можно ли использовать облачные сервисы вместо локального развертывания?

Ответ: Да. Это даже приветствуется. Опишите в отчете, какие облачные сервисы использовались и почему.

Вопрос: Нужно ли делать полноценный веб-интерфейс?

Ответ: Нет. Интерфейс может быть в виде API (FastAPI) или Jupyter Notebook с дашбордами.

Вопрос: Как показать Data Lineage без OpenMetadata?

Ответ: Можно использовать Apache Atlas, Amundsen или даже простую схему в отчете. Показать понимание происхождения данных.

Варианты кейс-задач для проектной работы

Кейс 1. Финансово-банковская аналитика

1.1. Система обнаружения дропперов (финансовое мошенничество)

Описание: банк нуждается в сервисе для выявления клиентов-дропперов – лиц, которые оформляют карты и передают их третьим лицам для вывода похищенных средств. Требуется разработать конвейер преобразования данных для агрегации транзакционных данных и построить модель, выявляющую подозрительные паттерны.

Источники данных: синтетические данные о транзакциях (можно сгенерировать по схеме банковских операций) или открытые наборы данных финансового мониторинга.

Набор данных для обнаружения мошеннических транзакций. Содержит данные о транзакциях и идентификационные данные пользователей – www.ieee-dataport.org.

Данные о прямых маркетинговых кампаниях португальского банка – archive.ics.uci.edu.

Архитектурные решения:

Компонент	Технология	Обоснование
Озеро данных	MinIO / HDFS	Хранение сырых журналов транзакций
Аналитическое хранилище	ClickHouse	Быстрые агрегации по миллионам транзакций
Реляционная база данных	PostgreSQL	Хранение справочников и витрин
Обработка	Apache Spark (PySpark)	Преобразования данных на больших объемах
Оркестрация	Apache Airflow	Ежедневное обновление конвейера

Связь с компетенциями:

- **BD-3.1** – проектирование гибридной модели хранения: PostgreSQL для оперативных транзакций, ClickHouse для аналитики, MinIO для озера данных;
- **BD-2.2** – подготовка транзакций для обучения модели выявления дропперов;
- **BD-4.1** – выбор Spark для распределенной обработки и ClickHouse для интерактивных запросов;
- **BD-4.3** – отчеты о качестве данных по транзакциям, прослеживаемость данных;
- **BD-5.1** – обоснование технологического стека под требования по задержке.

1.2. Система прогнозирования цен на недвижимость

Описание: на основе данных о характеристиках объектов недвижимости (местоположение, площадь, этаж, год постройки, инфраструктура) необходимо построить прогнозную модель стоимости квадратного метра для банковского проектного финансирования.

Источники данных: открытые наборы данных с объявлениями о продаже квартир (www.kaggle.com/c/house-prices-advanced-regression-techniques), данные о геолокации.

Архитектурные решения:

Компонент	Технология	Обоснование
Оперативная база данных	PostgreSQL	Транзакционное хранение данных
Хранилище данных	ClickHouse, dbt	Многослойная архитектура (промежуточный → преобразованный → витрины)
Обработка	Python, Pandas/Spark	Инженерия признаков (гео-представления)
Оркестрация	Apache Airflow	Конвейер преобразования данных из 5+ источников

Связь с компетенциями:

- **BD-3.1** – реализация многоуровневой архитектуры (сырые → промежуточные → витрины);
- **BD-2.2** – подготовка объектов по категориям для прогностической модели;
- **BD-4.1** – выбор dbt для преобразований в хранилище данных;
- **BD-4.3** – проверки качества данных (22 проверки в примере);
- **BD-5.1** – контейнерная оркестрация всей инфраструктуры.

Кейс 2. Аналитика городских данных

2.1. Рекомендательная система для выбора района проживания

Описание: разработать систему, которая на основе данных о преступности, демографии, инфраструктуре, предприятиях и погоде рекомендует пользователю оптимальный район для проживания с учетом его предпочтений.

Источники данных: открытые данные муниципальных служб, данные о погоде (<https://openweathermap.org/guide>), интерфейс прикладного программирования Google Maps (<https://developers.google.com/maps>).

Архитектурные решения:

Компонент	Технология	Обоснование
Озеро данных	S3/MinIO	Хранение разнородных наборов данных

Компонент	Технология	Обоснование
База данных	PostgreSQL	Нормализованное хранение
Слой интерфейса прикладного программирования	FastAPI	Доступ к данным для пользовательского интерфейса
Интерфейс искусственного интеллекта	Большая языковая модель, протокол управления моделями	Естественно-языковые запросы к базе данных
Визуализация	Jupyter/Панель управления	Интерактивная карта

Связь с компетенциями:

- **BD-3.1** – интеграция 5+ разнородных источников в единую схему;
- **BD-4.1** – выбор технологий для интерактивного анализа больших данных;
- **BD-4.3** – проверка качества данных на этапе загрузки;
- **BD-5.1** – проектирование архитектуры полного цикла данных;
- **PL-1.3** – разработка приложений интерфейса прикладного программирования;
- **SS-2.1** – представление результатов.

2.2. Система мониторинга городской экологии

Описание: создать конвейер для сбора и анализа метеоданных по городам России с возможностью прогнозирования температуры и контроля качества данных.

Источники данных: публичные интерфейсы прикладного программирования погоды (<https://data.gov/>), данные с датчиков.

Архитектурные решения:

Компонент	Технология	Обоснование
Хранилище	PostgreSQL, TimescaleDB	Хранение временных рядов
Архитектура	Многоуровневая	Многослойность данных
Оркестрация	Apache Airflow	Параллельный конвейер преобразования данных по городам
Модуль машинного обучения	LightGBM	Прогнозирование температуры
Интерфейс прикладного программирования	REST API	Программный доступ к данным

Связь с компетенциями:

- **BD-3.1** – выбор TimescaleDB для временных рядов;
- **BD-4.1** – параллельный конвейер преобразования данных с Airflow;
- **BD-4.3** – автоматический контроль качества данных;
- **BD-5.1** – развертывание одной командой Docker.

Кейс 3. Ретейл и маркетинг

3.1. Аналитическая платформа для прогнозирования оттока клиентов

Описание: телеком-оператор хочет прогнозировать отток клиентов. Требуется агрегировать данные из системы управления взаимоотношениями с клиентами, биллинга и журналов использования сервисов.

Источники данных: синтетические данные о клиентах телеком-компаний (<https://www.kaggle.com/datasets/rhonarosecortez/telco-customer-churn>).

Архитектурные решения:

Компонент	Технология	Обоснование
Оперативная база данных	PostgreSQL	Транзакционные данные
Документное хранилище	MongoDB	Поведенческие журналы
Обработка	PySpark	Преобразования данных
Качество данных	Great Expectations	Проверки качества
Оркестрация	Airflow	Ежедневные графы зависимостей задач

Связь с компетенциями:

- **BD-3.1** – гибридное хранилище (PostgreSQL, MongoDB);
- **BD-2.2** – подготовка данных по клиентам для обучения модели;
- **BD-4.1** – выбор Spark для конвейера преобразования данных;
- **BD-4.3** – внедрение идемпотентности конвейеров;
- **BD-5.1** – облачная инфраструктура (Yandex Cloud).

3.2. Система показателей эффективности для маркетинга

Описание: требуется построить конвейер для расчета маркетинговых показателей (стоимость привлечения клиента, окупаемость рекламных расходов) и их динамики в разрезе рекламных кампаний.

Источники данных: данные из рекламных кабинетов (интерфейсы прикладного программирования Яндекс.Директа, Google Ads), данные о продажах (<https://www.kaggle.com/datasets/instacart/instacart-market-basket-analysis>).

Архитектурные решения:

Компонент	Технология	Обоснование
Интеграция	N8n / Airflow	Автоматическая выгрузка из интерфейсов прикладного программирования
Хранилище данных	Google BigQuery / ClickHouse	Хранилище для аналитики
Аналитика	SQL, Python	Расчет показателей эффективности
Агент искусственного интеллекта	Большая языковая модель (интерфейс прикладного программирования)	Естественно-языковые запросы

Связь с компетенциями:

- **BD-4.1** – выбор технологий для инкрементальной загрузки;
- **BD-4.3** – проверки качества данных на этапах загрузки;
- **BD-5.1** – обоснование стека для конвейера преобразования данных;
- **PL-1.3** – разработка аналитических скриптов на Python;
- **SS-2.1** – создание интерфейса искусственного интеллекта для бизнес-пользователей.

Кейс 4. Поточковая обработка и аналитика в реальном времени

4.1. Аналитика журналов веб-сервиса в реальном времени

Описание: разработать конвейер для сбора, обработки и анализа журналов веб-сервиса: обнаружение аномалий, пиков нагрузки, популярных страниц в реальном времени.

Источники данных: синтетические журналы Nginx/Apache (генератор) или открытые наборы данных журналов (<https://www.kaggle.com/datasets/mahendradata/web-server-access-logs-labeled>).

Архитектурные решения:

Компонент	Технология	Обоснование
Потоковая передача	Apache Kafka	Буферизация событий
Обработка	Spark Streaming	Оконные агрегации
Хранилище	PostgreSQL, ClickHouse	История и реальное время
Оркестрация	Airflow	Запуск пакетных процессов
Визуализация	Metabase / Superset	Панели управления

Связь с компетенциями:

- **BD-4.1** – применение технологии потоковой обработки;
- **BD-4.3** – мониторинг качества потоковых данных;
- **BD-5.1** – выбор инфраструктуры для потоковой передачи;
- **PL-1.3** – разработка на PySpark.

Кейс 5. Биржевые и финансовые данные

5.1. Аналитический сервис для биржевых данных

Описание: создать систему для сбора, хранения и анализа биржевых данных (акции, индексы, дивиденды) с использованием конвейера преобразования данных и последующей визуализацией.

Источники данных: интерфейсы прикладного программирования Московской биржи (<https://www.moex.com/a7939>), Yahoo Finance (<https://rapidapi.com/mahmudulhasandev/api/yahoo-finance-stock-api>).

Архитектурные решения:

Компонент	Технология	Обоснование
Исторические данные	ClickHouse (аналитическое хранилище)	Аналитика больших объемов
Актуальные данные	PostgreSQL (оперативная база данных)	Быстрые запросы

Компонент	Технология	Обоснование
Оркестрация	Airflow	Параллельные графы зависимостей задач по сущностям

Связь с компетенциями:

- **BD-3.1** – разделение оперативной и аналитической баз данных для разных типов данных;
- **BD-4.1** – выбор ClickHouse для исторических данных;
- **BD-4.3** – проверки качества биржевых данных;
- **BD-5.1** – архитектура для масштабирования (WebSocket).

Чек-лист готовности к защите

Перед защитой проверьте:

- Все участники команды готовы отвечать на вопросы по своей части.
- Инфраструктура поднимается без ошибок (docker-compose up -d).
- DAG в Airflow работает без ошибок (зеленые таски).
- Данные загружены во все хранилища (PostgreSQL, ClickHouse, MinIO).
- ML-модель обучена и сохранена.
- Отчет полностью оформлен и сдан.
- Презентация готова и укладывается в 10-15 минут.
- Код загружен в репозиторий (GitHub/GitLab).

КРИТЕРИИ ОЦЕНИВАНИЯ ПРОЕКТНОЙ РАБОТЫ по дисциплине «Технологии хранения и обработки больших данных»

Максимальный балл за проект: 100 баллов

Структура оценки:

- Письменный отчет (30 баллов) – оценивается полнота и качество документации.
- Демонстрация прототипа (30 баллов) – оценивается работающее решение.
- Защита/презентация (20 баллов) – оценивается устная защита и ответы на вопросы.
- Командная работа (20 баллов) – оценивается вклад каждого участника.

Шкала перевода результата в баллы

Сумма баллов	Оценка	Условие
86-100	Отлично	Работа выполнена на высоком уровне, все требования реализованы в полном объеме, проявлена творческая инициатива, использованы дополнительные технологии. Студент демонстрирует глубокое понимание всех аспектов работы.
70-85	Хорошо	Работа выполнена в полном объеме, все требования реализованы, есть незначительные недостатки или недочеты. Студент демонстрирует хорошее понимание основных аспектов работы.
56-69	Удовлетворительно	Работа выполнена на минимально достаточном уровне, большинство требований реализованы, есть существенные недостатки. Студент демонстрирует базовое понимание работы, но допускает ошибки.

Сумма баллов	Оценка	Условие
0-55	Неудовлетворительно	Работа выполнена не в полном объеме, многие требования не реализованы. Студент не демонстрирует понимания ключевых аспектов работы.

КРИТЕРИИ ОЦЕНКИ

Критерий	Вес	Описание
Архитектура решения	20%	Насколько логично спроектирована архитектура, обоснован выбор технологий.
Реализация ETL/ELT	20%	Качество кода, корректность трансформаций, использование Spark и Airflow.
Хранение данных	25%	Корректность схемы БД, выбор моделей хранения, реализация загрузки данных.
Контроль качества	10%	Полнота DQ-проверок, качество DQ-отчета, Data Lineage.
ML-модель	10%	Качество признаков, обоснованность выбора модели, метрики.
Презентация и защита	10%	Качество презентации, аргументированность ответов, работа в команде.
Отчет	5%	Полнота и качество оформления отчета.

Информационные ресурсы

Официальная документация

1. Apache Software Foundation. Apache Spark: Unified Engine for Large-Scale Data Analytics [Электронный ресурс]. – URL: <https://spark.apache.org/docs/latest/> (дата обращения: 20.07.2026).
2. Apache Software Foundation. PySpark API Reference [Электронный ресурс]. – URL: <https://spark.apache.org/docs/latest/api/python/> (дата обращения: 20.07.2026).
3. Apache Software Foundation. Apache Iceberg: Open Table Format for Analytic Datasets [Электронный ресурс]. – URL: <https://iceberg.apache.org/docs/latest/> (дата обращения: 20.07.2026).
4. Apache Software Foundation. Apache Airflow: Platform to Programmatically Author, Schedule and Monitor Workflows [Электронный ресурс]. – URL: <https://airflow.apache.org/docs/> (дата обращения: 20.07.2026).
5. Apache Software Foundation. Apache Hadoop: Distributed Processing of Large Data Sets [Электронный ресурс]. – URL: <https://hadoop.apache.org/docs/> (дата обращения: 20.07.2026).
6. OpenMetadata Contributors. OpenMetadata Documentation: Metadata Management, Data Lineage and Data Quality [Электронный ресурс]. – URL: <https://open-metadata.org/docs> (дата обращения: 20.07.2026).

7. Docker Inc. Docker Documentation: Build, Ship and Run Distributed Applications [Электронный ресурс]. – URL: <https://docs.docker.com/> (дата обращения: 20.07.2026).

Фундаментальные основы и архитектура данных

1. Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка. – Астана: Sprint Book, 2024. – 640 с. – ISBN 978-601-08-3514-6.
Книга для понимания распределенных систем, моделей данных, репликации, партиционирования и транзакций. Дает теоретическую базу для выбора технологий и обоснования архитектурных решений.
2. Чернышов В. С. Большие данные. Принципы и практика построения масштабируемых систем обработки данных. – М.: ДМК Пресс, 2020. – 368 с. – ISBN 978-5-97060-802-9.
Рассматривается архитектура распределенных систем обработки данных, вопросы масштабирования и отказоустойчивости.
3. Кимбалл Р., Росс М. Инструментарий хранения и анализа данных. Полное руководство по размерному моделированию. – М.: Бомбора, 2021. – 656 с. – ISBN 978-5-04-118211-9.
Проектирование хранилищ данных и моделирование. Дает понимание архитектуры слоев (ODS, DDS, витрины) и звездообразных схем.

Ключевые технологии обработки и хранения данных

Apache Spark

1. Карау Х., Конвински Э., Венделл П., Захария М. Изучаем Spark. Молниеносный анализ данных. – М.: ДМК Пресс, 2015. – 400 с. – ISBN 978-5-97060-230-0.
Классическое руководство по Apache Spark. Содержит примеры на Python и Scala. Поможет разобраться с API DataFrames, трансформациями, оптимизацией запросов и работой с большими данными.
2. Неверов Е. В. Spark и Hadoop. Обработка больших данных. – М.: Горячая линия – Телеком, 2020. – 350 с. – ISBN 978-5-9912-0854-8.
Рассматривает совместное использование Spark и Hadoop, практические аспекты настройки кластеров.
3. Дьяков С. Г. Анализ данных с помощью Apache Spark. – М.: Национальный Открытый Университет «ИНТУИТ», 2020. – 320 с. – ISBN 978-5-4497-0902-8.
Учебное пособие по анализу данных с использованием Spark, охватывает основные операции и оптимизацию.

Apache Iceberg и форматы хранения

4. Хватов А. И. Apache Iceberg для профессионалов: современный табличный формат для аналитических данных. – М.: ДМК Пресс, 2024. – 280 с. – ISBN 978-5-93700-210-5.
Книга по табличному формату Iceberg. Описывает его преимущества (ACID, Time Travel, Schema Evolution), архитектуру и практики использования.
5. Соснин В. В., Балакшин П. В., Шилко Д. С. и др. Введение в параллельные вычисления. – СПб: Университет ИТМО, 2023. – 128 с. – URL: <https://books.ifmo.ru/file/pdf/3230.pdf>.
Учебное пособие по параллельным вычислениям, включая распределенное хранение данных.

NoSQL и распределенные базы данных

6. Мартишин С. А., Симонов В. Л., Храпченко М. В. Базы данных: Работа с распределенными базами данных и файловыми системами на примере MongoDB и HDFS

с использованием Node.js, Express.js, Apache Spark и Scala. – М.: Форум: ИНФРА-М, 2023. – 235 с. – ISBN 978-5-16-015133-5.

Учебное пособие, охватывающее работу с распределенными базами данных и файловыми системами на примере MongoDB и HDFS.

7. Волк В. К., Осеев В. Ю., Черепанов О. С. Базы данных: Учебник. – Вологда: Инфра-Инженерия, 2025. – 400 с. – ISBN 978-5-9729-2232-7.

Книга охватывает широкий круг вопросов, связанных с технологиями хранения и управления данными. Рассматриваются как классические реляционные системы, так и постреляционные NoSQL-решения.

8. Синха Д. Экосистема Hadoop, YARN, NoSQL, MongoDB, Spark и Scala. – Кишинёв: Sciencia Scripts, 2024. – 72 с. – ISBN 978-620-7-67611-8.

Руководство по экосистеме Hadoop, включая HDFS, MapReduce, YARN, NoSQL, MongoDB и Spark.

Обработка больших данных

1. Марц Н., Уоррен Д. Большие данные. Принципы и практика построения масштабируемых систем обработки данных в реальном времени. – М.: Вильямс, 2018. – 368 с. – ISBN 978-5-907114-02-9.

Рассматриваются принципы построения масштабируемых систем обработки данных в реальном времени.

2. Картер Д. Обработка больших данных. – М.: ЛитРес: Самиздат, 2024. – 320 с. – ISBN 978-5-04-209958-0.

Пособие для изучения технологий больших данных, охватывающее экосистему Hadoop (HDFS, MapReduce, Hive, Pig, HBase), Apache Spark (RDD, DataFrames, Spark SQL, Spark Streaming, MLLib) и Apache Kafka.

3. Таненбаум Э. Распределенные системы. Принципы и парадигмы. – М.: Питер, 2003. – 876 с. – ISBN 5-272-00053-6.

Учебник по распределенным системам, охватывающий принципы организации распределенных вычислений и хранения данных.